

Apex Project 5 Performance Report

By: Lucas Chaparro and Narendra Nath Chalasani

We ran 2 separate queries in order to measure the performance difference between them. The first was a sequential scan, and the second was an index scan. The sequential scan was slower, and the index scan was faster in terms of execution.

To start, we seeded the database with 1 million mock ticket records to produce a functioning dataset. Within the PostgreSQL database, we the execution of the EXPLAIN ANALYZE command was necessitated to capture the query lookup strategy and runtime of the query.

```
Administrator: Command Prompt - docker compose exec psql_db psql -U admin -d apex_events
C:\Users\naren\Downloads\Team 8 - Project 5>docker compose exec psql_db psql -U admin -d apex_events
psql (15.18)
Type "help" for help.

apex_events=# EXPLAIN ANALYZE SELECT * FROM tickets WHERE status = 'active' AND event_id = 123;
               QUERY PLAN
-----
Gather  (cost=1000.00..16261.10 rows=1 width=39) (actual time=125.282..129.601 rows=0 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  -> Parallel Seq Scan on tickets  (cost=0.00..15261.00 rows=1 width=39) (actual time=116.326..116.327 rows=0 loops=3)
       Filter: (((status)::text = 'active'::text) AND (event_id = 123))
       Rows Removed by Filter: 333333
Planning Time: 3.388 ms
Execution Time: 129.690 ms
(8 rows)

apex_events=# CREATE INDEX ix_ticket_status_event_id ON tickets(status,event_id);
CREATE INDEX
apex_events=# EXPLAIN ANALYZE SELECT * FROM tickets WHERE status = 'active' AND event_id = 123;
               QUERY PLAN
-----
Index Scan using ix_ticket_status_event_id on tickets  (cost=0.42..7.88 rows=1 width=39) (actual time=0.080..0.081 rows=0 loops=1)
  Index Cond: (((status)::text = 'active'::text) AND (event_id = 123))
Planning Time: 0.773 ms
Execution Time: 0.145 ms
(4 rows)
```

By writing the command “CREATE INDEX ix_ticket_status_event_id ON tickets(status, event_id);” it created an index stored in the memory that can be accessed much quicker instead of having to inspect every single ticket to find the ones that have a specific value. This builds a B-Tree Composite Index that structures the status and event_id values into a tree that can be quickly searched. PostgreSQL can quickly access the correct branch which drops the lookup time to $O(\log N)$ time complexity using an index scan, which is not possible with a sequential scan. 1e400645e8d9_add_composite_index.py is the Alembic migration used for this test. It injects a B-Tree Composite Index for the parameters status and event_id.

Sequential Scan Result

Metric	Value	Status
Query Plan	Gather -> Parallel Seq Scan on tickets	Bottleneck
Rows Removed by Filter	333,333 (per worker)	
Planning Time	3.388 ms	
Execution Time	129.690 ms	FAILS <100ms target

Index Scan Result

Metric	Value	Status
Query Plan	Index Scan using ix_ticket_status_event_id	Optimized
Index Condition	status = 'active' AND event_id = 123	
Planning Time	0.773 ms	
Execution Time	0.145 ms	PASSES <100ms target